SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Sowmiya. D (192511063) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sowmiya. D

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Experiment 1: Decision Tree Classification

Objective

Implement and evaluate a Decision Tree classifier using the **Iris dataset**.

Libraries Required

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

(a) Load, preprocess and split the dataset

```
# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target column
df["target"] = iris.target

# Add target names for easier interpretation
df["species"] = df["target"].map({
    0: "setosa",
    1: "versicolor",
    2: "virginica"
})

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Features and target
X = df[iris.feature_names]
y = df["target"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.30,
    random_state=0,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected first 5 rows

sepal length	sepal width	petal length	petal width	target	species
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

There are **no missing values**, so no imputation is required. The target classes have been encoded numerically as 0, 1 and 2.

(b) Build the Decision Tree

Here, **Information Gain (entropy)** is used as the splitting criterion.

```
# Create Decision Tree classifier
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
print("Decision Tree Structure:\n")
print(export_text(model, feature_names=list(X.columns)))

# Display feature importance
print("\nFeature Importance:")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(f"{feature}: {importance:.4f}")

# Plot the tree
plt.figure(figsize=(16, 10))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
```

```
plt.title("Decision Tree - Iris Dataset")
plt.show()
```

Root Node

For this particular train/test split, the **root node is:**

petal length (cm) ≤ 2.35

The tree first divides the observations according to petal length.

The feature importances are approximately:

sepal length (cm): 0.0223

sepal width (cm): 0.0649

petal length (cm): 0.5869

petal width (cm): 0.3259

Therefore, **petal length has the highest importance (≈58.7%)**.

Justification

The root node is petal length <= 2.35 because the decision-tree algorithm found this split to provide the **largest reduction in entropy**, i.e. the highest information gain among the candidate splits at the root.

This is also supported by the high feature importance of petal length.

(c) Model Evaluation

```
# Make predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification report
print("\nClassification Report:")
print(
   classification_report(
      y_test,
      y_pred,
      target_names=iris.target_names
   )
)
```

For the specified split (random_state=0), the accuracy is approximately:

Accuracy: 0.9111

So the model correctly classifies approximately **91.11%** of the test samples.

Misclassified instances

```
# Display misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = [
   iris.target_names[i] for i in y_test[y_test != y_pred]
```

```
]

misclassified["Predicted"] = [
    iris.target_names[i] for i in y_pred[y_test != y_pred]
]

print("\nMisclassified Instances:")
print(misclassified)
```

For this split, four misclassified observations are obtained. Two examples are:

Index	Actual	Predicted
134	virginica	versicolor
68	versicolor	virginica

Two additional misclassifications are:

Index	Actual	Predicted
54	versicolor	virginica
129	virginica	versicolor

Performance comment

The Decision Tree performs well, achieving about **91% accuracy**. Setosa is particularly easy to distinguish because its petal measurements are substantially different from the other two species. Most errors occur between **versicolor and virginica**, whose measurements overlap considerably.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

Implement Q-Learning in a **4 × 4 Grid World**, learn the optimal policy, and observe the evolution of the Q-table.

(a) Define the Grid World

We use:

- Grid size: **4 × 4**
- Actions: **Up, Down, Left, Right**
- Goal reward: **+10**
- Normal movement: **−1**
- Obstacle: **−5**
- Learning rate α = **0.1**
- Discount factor γ = **0.9**
- Exploration rate ε = **0.2**

The grid is:

```
S . . .
. X . .
. . X .
. . . G
```

Where:

- S = Start
- G = Goal
- X = Obstacle
- . = Normal state

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Grid dimensions
ROWS = 4
COLS = 4

# States
states = list(range(ROWS * COLS))

# Actions
actions = ["Up", "Down", "Left", "Right"]
action_to_index = {
   "Up": 0,
   "Down": 1,
   "Left": 2,
   "Right": 3
}

# Start and goal
START = 0
GOAL = 15

# Obstacles
OBSTACLES = {5, 10}

# Hyperparameters
alpha = 0.1       # Learning rate
gamma = 0.9       # Discount factor
epsilon = 0.2     # Exploration rate

# Number of episodes
episodes = 100

# Initialize Q-table with zeros
Q = np.zeros((ROWS * COLS, len(actions)))

print("Initial Q-table:")
print(Q)

print("\nHyperparameters:")
print("Alpha =", alpha)
print("Gamma =", gamma)
print("Epsilon =", epsilon)
```

Environment Functions

```
def state_to_position(state):
   return divmod(state, COLS)


def position_to_state(row, col):
```

```
    return row * COLS + col


def step(state, action):
    """
    Perform one action in the grid.
    Returns: next_state, reward, done
    """

    row, col = state_to_position(state)

    # Calculate new position
    if action == "Up":
        new_row, new_col = row - 1, col
    elif action == "Down":
        new_row, new_col = row + 1, col
    elif action == "Left":
        new_row, new_col = row, col - 1
    elif action == "Right":
        new_row, new_col = row, col + 1

    # Boundary handling
    if (
        new_row < 0 or
        new_row >= ROWS or
        new_col < 0 or
        new_col >= COLS
    ):
        return state, -1, False

    next_state = position_to_state(new_row, new_col)

    # Goal
    if next_state == GOAL:
        return next_state, 10, True

    # Obstacle
    if next_state in OBSTACLES:
        return next_state, -5, False

    # Normal step
    return next_state, -1, False
```

(b) Run Q-Learning

The Q-learning update rule is:

$$
Q(s,a) \leftarrow Q(s,a) +
\alpha [r+\gamma \max_{a'}Q(s',a')-Q(s,a)]
$$

```
# Reset Q-table
```

```
Q = np.zeros((ROWS * COLS, len(actions)))

# Store cumulative rewards
episode_rewards = []

# Store Q-values at episodes 1, 50 and 100
q_snapshots = {}

# Representative states
representative_states = [0, 6]

for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_number in range(100):

        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            action_index = np.random.randint(len(actions))
        else:
            action_index = np.argmax(Q[state])

        action = actions[action_index]

        # Take action
        next_state, reward, done = step(state, action)

        # Q-learning update
        best_next_q = np.max(Q[next_state])

        Q[state, action_index] = Q[state, action_index] + alpha * (
            reward +
            gamma * best_next_q -
            Q[state, action_index]
        )

        total_reward += reward

        state = next_state

        if done:
            break

    episode_rewards.append(total_reward)

    # Record snapshots
    if episode in [1, 50, 100]:
```

```
        q_snapshots[episode] = Q[representative_states].copy()


# Display Q-values
for episode, values in q_snapshots.items():

    print(f"\nQ-values at Episode {episode}:")

    for state, q_values in zip(representative_states, values):
        print(f"State {state}:")
        for action, value in zip(actions, q_values):
            print(f"  {action}: {value:.4f}")
```

Q-table evolution

The following code presents the Q-values for two representative states at Episodes 1, 50 and 100:

```
for episode in [1, 50, 100]:

    print("\n" + "=" * 40)
    print(f"Episode {episode}")
    print("=" * 40)

    for state in representative_states:

        print(
            f"State {state}: "
            f"{q_snapshots[episode][
                representative_states.index(state)
            ]}"
        )
```

Initially, all Q-values are zero. As the agent interacts with the environment, rewards are propagated backward through the Q-table.

Therefore:

Episode 1 → Q-table mostly near zero

Episode 50 → Useful action values begin to emerge

Episode 100 → Values corresponding to good paths become dominant

---

(c) Extract the Final Learned Policy

```
# Convert Q-table into policy
policy = []

for state in states:

    if state == GOAL:
        policy.append("G")

    elif state in OBSTACLES:
        policy.append("X")

    else:
        best_action = actions[np.argmax(Q[state])]
        policy.append(best_action)
```

```
# Display policy as a 4x4 grid
print("Final Learned Policy:\n")

for row in range(ROWS):

    row_policy = []

    for col in range(COLS):

        state = position_to_state(row, col)
        row_policy.append(policy[state])

    print(row_policy)
```

A typical learned policy will direct the agent toward the goal while avoiding the obstacles.
For example, conceptually:
Right Right Right Down
Down X Down Down
Down Right X Down
Right Right Right G
The exact learned policy can vary slightly because Q-learning uses random exploration.

Plot Cumulative Reward per Episode

```
plt.figure(figsize=(10, 5))

plt.plot(episode_rewards)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")

plt.grid(True)
plt.show()
```

To make the convergence trend easier to observe, we can also plot a moving average:

```
window = 10

moving_average = np.convolve(
    episode_rewards,
    np.ones(window) / window,
    mode="valid"
)

plt.figure(figsize=(10, 5))

plt.plot(
    range(window, episodes + 1),
    moving_average
)
```

```
plt.xlabel("Episode")
plt.ylabel("Average Cumulative Reward")
plt.title("Q-Learning Convergence")

plt.grid(True)
plt.show()
```

Convergence behaviour

At the beginning, the agent explores the environment and frequently takes inefficient actions. Consequently, the cumulative reward is relatively low.

As training progresses:

1. The agent discovers rewarding paths.
2. Q-values for useful actions increase.
3. The agent increasingly selects actions leading toward the goal.
4. The cumulative reward generally improves.
5. Eventually, the policy becomes relatively stable.

Thus, the increasing/stabilizing reward curve indicates that the **Q-learning algorithm is converging toward a useful policy**.

Because ε is kept at 0.2, the agent continues to explore 20% of the time, so the reward curve may still contain some fluctuations even after learning.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation